Introduction

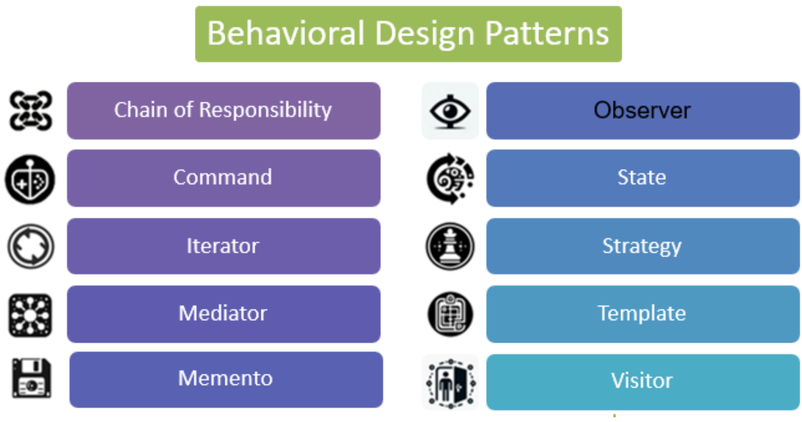
Behavioral Patterns

Behavioral design patterns are a subset of software design patterns concerned with assigning responsibilities among objects and how these objects communicate and interact. They help to manage complex flows and interactions in software systems, creating a more modular, scalable, and maintainable system.

By using the Behavioral Design Patterns, you can:

- enable flexible and efficient object interaction
- reduce tight coupling between classes for better maintainability
- simplify complex communication and collaboration between objects
- allow dynamic modification of behavior at runtime.

Types of Behavioral Design Patterns



Types of Behavioral Patterns

Pattern	Introduction
Chain of Responsibility	Delegates commands to a chain of processing objects.
Command	Encapsulates a command request as an object.
Interpreter	Implements a specialized language interpretation.
Iterator	Sequentially accesses elements of a collection.
Mediator	Centralizes complex communications and control between related objects.
Memento	Captures and externalizes an object's internal state.
Observer	Maintains consistency between loosely coupled objects.
State	Allows an object to change its behavior when its internal state changes.
Strategy	Enables an algorithm's behavior to be selected at runtime.
Template Method	Defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps.
Visitor	Defines a new operation to a class without change.